

The Phoenix

Concurrent viewers, counted correctly. Peak and average at minute, hour and day grain, across nine filter dimensions, from a delta table that never rescans raw history.

905,558

GRADED CORPUS EVENTS
the-phoenix.cricheroes.io

7,000,000

UNSEEN DAY, SAME QUERIES

0

DIFFS AGAINST BRUTE-FORCE ORACLE

Interval overlap is not concurrency

A session can be open while the app is backgrounded, the player is paused, or the heartbeat has simply stopped arriving. Counting that time as watching overstates the audience, and every decision made on the dashboard inherits the error.

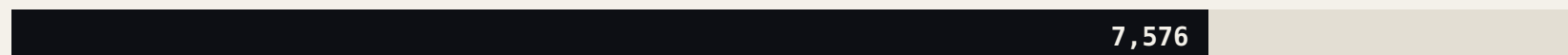
Naive session-span counting

peak concurrent



Foreground-only, active interval

peak concurrent



A **31 percent** overstatement, and 1,592 minutes where the naive method invents an audience that was never there. That gap is the entire problem, removed and measured.

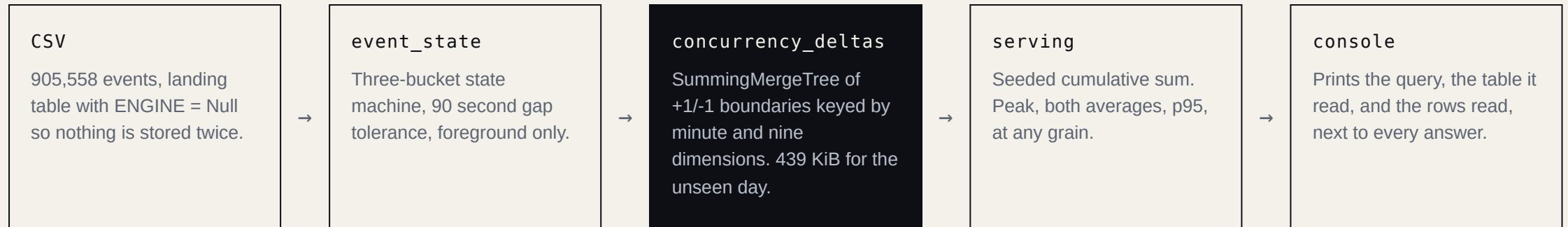
Model the active interval, not the session

Every event moves a session between three states: foreground, background, and gone. State holds for a 90 second tolerance and no longer, so a session that stops emitting stops counting whether or not it ever formally ended.

Those active intervals normalise to per-session minute runs, then to +1/-1 boundary deltas. Concurrency at any minute is a running sum, so the serving layer reads boundaries rather than history.

- **Peak is computed, never stored.** A platform slice and a platform-plus-country slice peak at different minutes inside the same range, so a pre-rolled peak per dimension is wrong by construction.
- **Both averages are reported.** 88.06 over all 1,440 minutes, 200.00 over the 634 minutes with an audience. Averaging sparse rows instead returns 246.5 against a true 88.2.
- **Open sessions retract.** A session still open when the question is asked is counted, then corrected as its real ending arrives.

Raw history is read once, on the way in



Nothing in the serving path touches `raw_events`. The gate that proves it reads ClickHouse's own `query_log` and fails the build if a served query ever does.

The numbers are not asserted

CLAIM	NUMBER	REPRODUCE
Serving layer versus brute-force oracle	3,663 minutes, 0 diffs	parity.sh
Open sessions absorbed incrementally	5,316 minutes, 0 diffs	test_open_sessions.sh
Full rebuild run twice, derived tables diffed	0 diff lines, 5 of 5 tables	prove_idempotence.sh
Frozen slice stable under concurrent writes	34 metrics, 0 differing lines	frozen_gate.sh
Peak concurrent sessions, graded corpus	2,828 at 10:56	ground_state.sh
Average over all 1,440 minutes	88.06	bench.sh
Average over the 634 minutes with an audience	200.00	bench.sh

Every figure resolves through `evidence/LEDGER.tsv` to the command that produced it and the stamped artifact it wrote. `check_docs.sh` fails the build when a claim and its evidence disagree.

Fast because it reads less, not because it is bigger

12 ms

WORST-SHAPE QUERY,
30,662 ROWS AGAINST A BUDGET OF 80,712

2 of 4

GRANULES TOUCHED BY A
PLATFORM FILTER, 16,384 ROWS

70 s

CSV TO VERIFIED SERVING LAYER,
FULL REBUILD FROM COLD

The sorting key is ordered by pruning power, so the dimensions a judge actually filters on come first. The honest caveat, stated in the README rather than hidden: a ninth-position dimension such as `video_resolution` does not prune, and reads 354,185 rows against 354,305 unfiltered.

Released after the pipeline was built. Same queries, untouched.

7,000,000

EVENTS FOR 2026-07-31,
7.7X THE GRADED CORPUS

41 s

TO DERIVE, ALL
INVARIANTS PASS

22,416

PEAK CONCURRENT SESSIONS
AT 11:16

914.65

DAILY AVERAGE OVER
1,440 MINUTES

Both datasets sit behind a switch in the console, so the generalisation can be seen rather than claimed: the original corpus every number was measured against, and the unseen day, side by side, without a restart.

Nine dimensions cost 98 KiB and change no answer

The same 7,000,000 row day, derived twice: five dimensions against nine. The widened key was accepted only because the comparison came back identical, line for line.

METRIC	FIVE DIMENSIONS	NINE DIMENSIONS
Peak concurrent	22,416 at 11:16	22,416 at 11:16
Average, all minutes	914.65	914.65
Average, active minutes	1,667.21	1,667.21
Asserted runs	118,498	118,498
Derive time	41 s	41 s
Delta table on disk	341 KiB	439 KiB

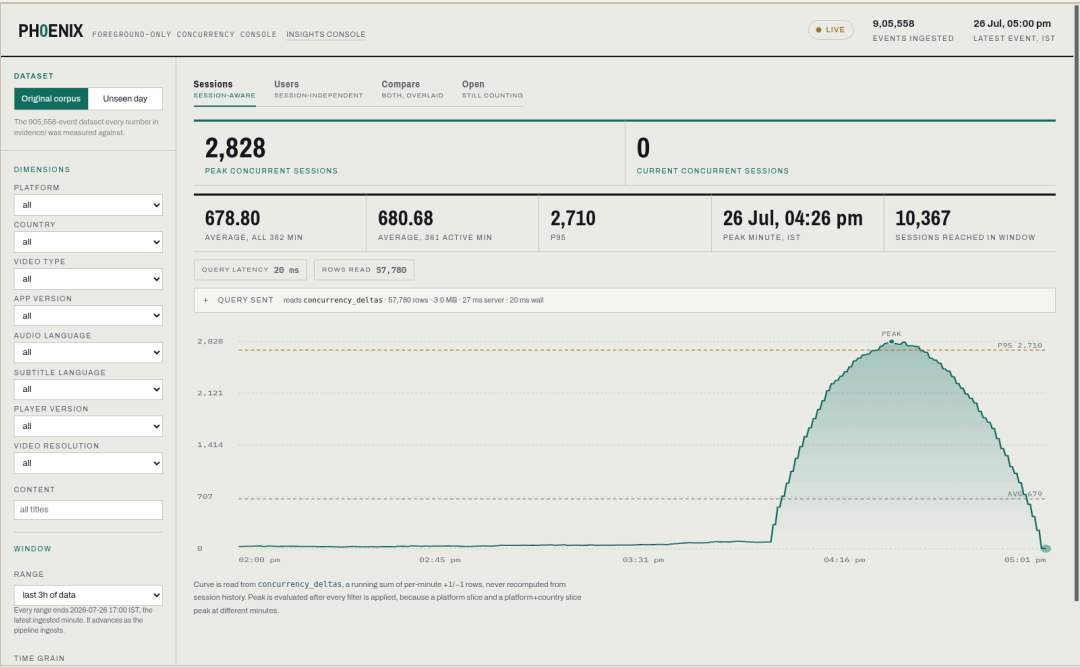
Key cardinality rises 1.6x, realised rows only 11 percent, because SummingMergeTree collapses what the wider key splits.

An evidence ledger, not a claims list

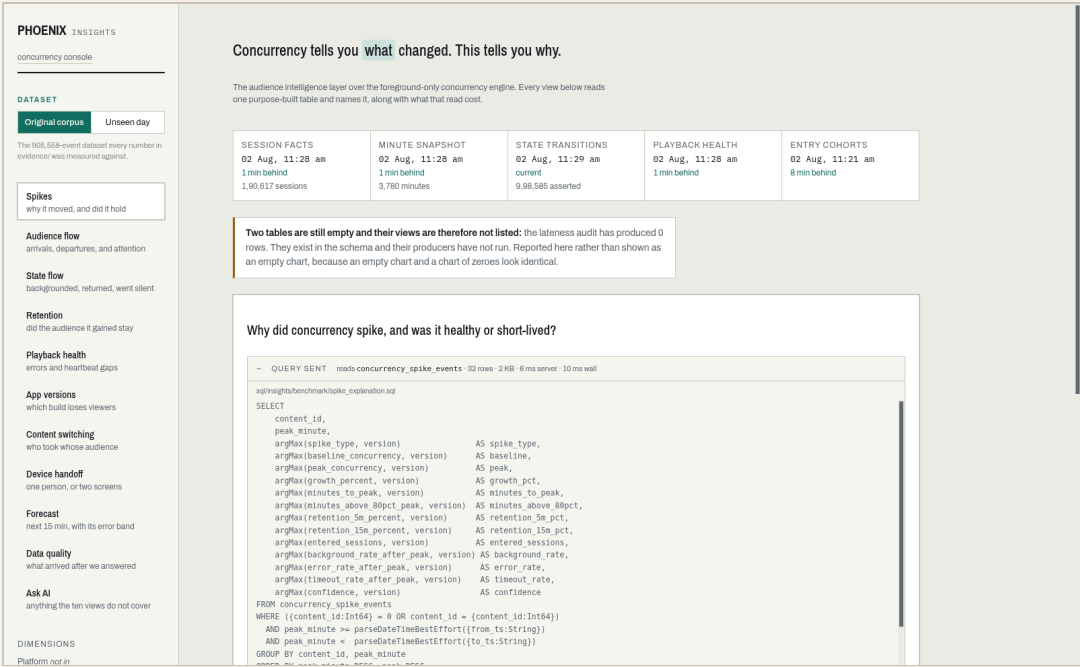
- **Every script writes a stamped TSV.** Run timestamp, git SHA, host, row count, watermark. A claim with no artifact behind it fails `check_docs.sh`.
- **Gates read ClickHouse's own query_log** to prove which tables a served query touched, rather than trusting the SQL to say so.
- **Corrections are kept, not erased.** A positional INSERT silently shifted a column while all three derive invariants passed. That is written down, with the fix: explicit column lists everywhere.
- **Two consoles on one port.** The graded concurrency console, and a live insight console where the curve builds in near real time from a running producer.
- **A live agent, not a canned demo.** The Ask tab is backed by a LibreChat agent with a real ClickHouse MCP tool, so an open-ended question is answered by querying, not by a script.
- **Deployed behind one published port.** A check fails the deploy if anything else is exposed.

THE SYSTEM, RUNNING

Both consoles, live on one port



v1, the concurrency console. Nine filter dimensions, the dataset switch, and the query that produced the number printed beside it: 57,780 rows, 20 ms.



v2, the insight console. Ten purpose-built views over a live database, each naming its table and read cost. Empty tables are declared, not drawn as zero.

Ask it in English, and watch what it ran

- **LibreChat, not a chat widget.** The Ask tab is a LibreChat agent served behind the same published port. nginx resolves the upstream lazily, so the graded concurrency console stays up even when chat is not running.
- **One tool, read-only.** A ClickHouse MCP tool over the same delta tables the console reads. The agent writes the SQL and the answer comes back from a query, not from the model.
- **Langfuse on every turn.** LibreChat's native Langfuse tracing is on, so each Ask turn emits a trace carrying the prompt, the tool call, the SQL issued, the rows read and the latency. An agent answer is auditable the same way a console answer is.

PHOENIX INSIGHTS

concurrency console

DATASET

Original corpusUnseen day

The 905,568-event dataset every number in evidence was measured against.

Spikes

why it moved, and did it hold

Audience flow

arrivals, departures, and attention

State flow

backgrounded, returned, went silent

Retention

did the audience it gained stay

Playback health

errors and heartbeat gaps

App versions

which build loses viewers

Content switching

who took whose audience

Device handoff

one person, or two screens

Forecast

next 15 min, with its error band

Data quality

what arrived after we answered

Ask AI

anything the ten views do not cover

Concurrency tells you what changed. This tells you why.

The audience intelligence layer over the foreground-only concurrency engine. Every view below reads one purpose-built table and names it, along with what that read cost.

SESSION FACTS	MINUTE SNAPSHOT	STATE TRANSITIONS	PLAYBACK HEALTH	ENTRY COHORTS
02 Aug, 11:29 am 1 min behind 1,92,577 sessions	02 Aug, 11:29 am 1 min behind 3,781 minutes	02 Aug, 11:30 am current 9,99,091 asserted	02 Aug, 11:29 am 1 min behind	02 Aug, 11:22 am 8 min behind

Two tables are still empty and their views are therefore not listed: the lateness audit has produced 0 rows. They exist in the schema and their producers have not run. Reported here rather than shown as an empty chart, because an empty chart and a chart of zeroes look identical.

Why did concurrency spike, and was it healthy or short-lived?

— QUERY SENT

reads concurrency_spike_events - 34 rows - 2 KB - 8 ms server - 11 ms wall

sqlnightsabenchmark/spike_explanation.sql

SELECT

content_id,

peak_minute,

argMax(spike_type, version) AS spike_type,

argMax(baseline_concurrency, version) AS baseline,

argMax(peak_concurrency, version) AS peak,

argMax(growth_percent, version) AS growth_pct,

argMax(minutes_to_peak, version) AS minutes_to_peak,

argMax(minutes_above_80pct_peak, version) AS minutes_above_80pct,

argMax(retention_5m_percent, version) AS retention_5m_pct,

argMax(retention_15m_percent, version) AS retention_15m_pct,

argMax(entered_sessions, version) AS entered_sessions,

argMax(background_rate_after_peak, version) AS background_rate,

argMax(error_rate_after_peak, version) AS error_rate,

argMax(timeout_rate_after_peak, version) AS timeout_rate,

argMax(confidence, version) AS confidence

FROM concurrency_spike_events

The Ask tab. An open-ended question resolved by the agent against the delta tables, with the tool call and the query it issued shown beside the answer.

LIVE NOW

See it answer

the-phoenix.cricheroes.io serves the graded console at / and the live insight console at /v2.
Both print the query, the table, and the rows read beside every answer, so nothing on screen has to be taken on trust.

31%

OVERSTATEMENT REMOVED
AND MEASURED
The Phoenix · SonyLIV Click-a-thon

0

DIFFS AGAINST THE ORACLE,
BATCH AND INCREMENTAL

12 ms

WORST-SHAPE QUERY,
NINE FILTER DIMENSIONS

41 s

TO ABSORB AN UNSEEN
SEVEN MILLION EVENT DAY